WEB APPLICATION SECURITY ASSESSMENT

CROSS-SITE REQUEST FORGERY

CSRF with No Defenses

ASSESSMENT	CSRF vulnerability with no defenses
TARGET FUNCTION	Authenticated account email change
VULNERABILITY	Cross-Site Request Forgery (CSRF)
AUTHOR	Joshua Sanctus
DATE	8 November 2026

Prepared for technical learning and portfolio documentation.
Testing was limited to the authorised training environment.

Contents

1 Executive Summary 3

2 Scope and Lab Details 3

3 Vulnerability Overview 3

4 Methodology 3

5 Proof of Concept 4

6 Evidence and Results 4

7 Security Impact 5

8 Root Cause 5

9 Remediation 6

10 Conclusion 6

11 Limitations 6

1 Executive Summary

This report documents completion of the PortSwigger Web Security Academy lab *CSRF vulnerability with no defenses*. The lab demonstrates a Cross-Site Request Forgery weakness in an authenticated account email-change function. The supplied evidence shows the account page, an exploit-server HTML form configured to submit an email-change request automatically, and the platform's successful-lab confirmation.

The demonstrated impact is an unauthorised change to the victim account's email address when the victim is authenticated and induced to visit the attacker-controlled exploit page. The lab was solved successfully.

> **Assessment result**
> A cross-site HTML form submitted a POST request to the account email-change endpoint. The request changed the account email address, and PortSwigger confirmed successful completion of the lab.

2 Scope and Lab Details

Platform	PortSwigger Web Security Academy
Lab	CSRF vulnerability with no defenses
Vulnerability class	Cross-Site Request Forgery (CSRF)
Affected function	Authenticated account email-change operation
Evidence	Three screenshots supplied for this assessment
Scope	Assigned, intentionally vulnerable lab instance

Testing was limited to the authorised training environment.

3 Vulnerability Overview

Cross-Site Request Forgery occurs when a web application accepts a state-changing request based on the browser's automatically attached authentication context without adequately verifying that the request was intentionally initiated by the user through the legitimate application.

A malicious website may cause a victim's browser to submit a request to another website where the victim is already authenticated. If the target application does not enforce effective CSRF protections, it may process that request using the victim's session.

In this lab, the account email-change operation is vulnerable because a cross-site form submission can change the email address without a CSRF token or another effective request-origin defence.

4 Methodology

1. Accessed the account page while authenticated and identified the email-change functionality.

2. Inspected the form/request structure and identified the email address as a form parameter.
3. Created an HTML form on the PortSwigger exploit server that submits a POST request to the account email-change endpoint.
4. Configured the form with a hidden email field and JavaScript that automatically submits the form when the exploit page is loaded.
5. Used the exploit-server workflow to deliver the page to the lab victim.
6. Observed that the account email address changed and confirmed the lab completion banner.

5 Proof of Concept

The exploit used a cross-site HTML form with a POST method. The form contained a hidden email field and JavaScript to submit the form automatically.

```
<form method="POST" action="https://YOUR-LAB-ID.web-security-academy.net/
   my-account/change-email">
    <input type="hidden" name="email" value="attacker@example.com">
</form>
<script>
    document.forms[0].submit();
</script>
```

Replace the lab host and email value with the values used in the authorised lab instance. The supplied exploit-server screenshot shows a hidden email parameter and an automatic form submission script. The report does not reproduce the unique lab identifier or the full HTTP request headers.

6 Evidence and Results

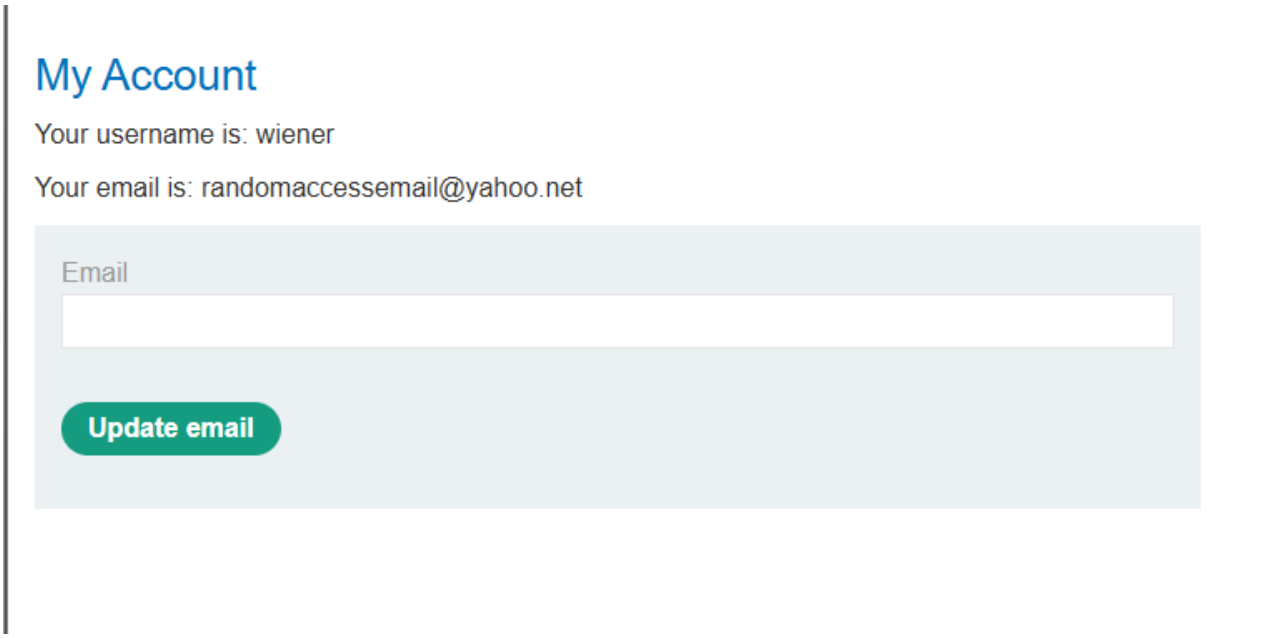


Figure 1: Authenticated account page showing the account username, current email address and email-update form.

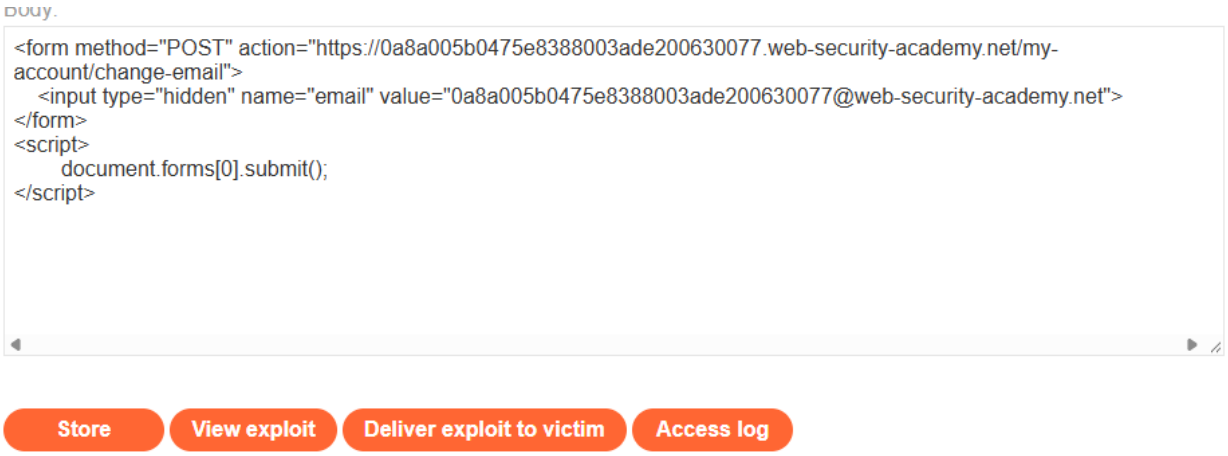


Figure 2: Exploit-server HTML body containing a POST form with a hidden email field and automatic JavaScript form submission.



Figure 3: PortSwigger confirmation banner indicating that the CSRF lab was solved.

7 Security Impact

In a real application, successful CSRF may allow an attacker to trigger state-changing actions using the victim's authenticated browser session. Depending on the application's functionality, examples may include changing account details, modifying security settings or performing transactions.

In this lab, the demonstrated action was changing the victim account's email address. The supplied evidence does not establish broader account takeover or access to the victim's email inbox.

8 Root Cause

The vulnerable operation did not enforce an effective anti-CSRF mechanism for the state-changing request. The exploit succeeded through a cross-site form submission, demonstrating that the application did not adequately verify that the email-change request originated from a trusted interaction with the application.

9 Remediation

1. **CSRF tokens:** Use unpredictable, session-bound tokens and validate them server-side on every state-changing request.
2. **SameSite cookies:** Configure the `SameSite` attribute appropriately (typically `Lax` or `Strict` where compatible) as defence in depth. Do not rely on it as the sole protection.
3. **Origin validation:** Validate the `Origin` header for state-changing requests. Use `Referer` validation as a carefully designed fallback where appropriate.
4. **Sensitive changes:** Require re-authentication or additional confirmation for sensitive account changes, such as changing an email address.
5. **HTTP method design:** Ensure GET requests do not perform state-changing operations.
6. **Consistent enforcement:** Apply protections to all state-changing endpoints, including API routes.

10 Conclusion

The lab demonstrated a CSRF vulnerability in an authenticated email-change function. A cross-site HTML form automatically submitted a POST request, resulting in an email-address change and successful lab completion. The primary remediation is server-side CSRF-token validation, supported by appropriate cookie and request-origin controls.

11 Limitations

This report is based on the three supplied screenshots. They show the account page, the exploit form and the solved confirmation. They do not include a complete captured HTTP request/response pair, server-side source code or an independent test of each potential impact.

No CVSS score or CVE is assigned because this is a deliberately vulnerable training lab rather than a separately assessed production vulnerability.

> **Assessment boundary**
> This is an intentionally vulnerable training exercise, not a production-system assessment. The report should be shared with its lab context clearly identified.

END OF REPORT